

Architecture & Design Decisions

How Clean Architecture and MVVM are applied across the project layers.

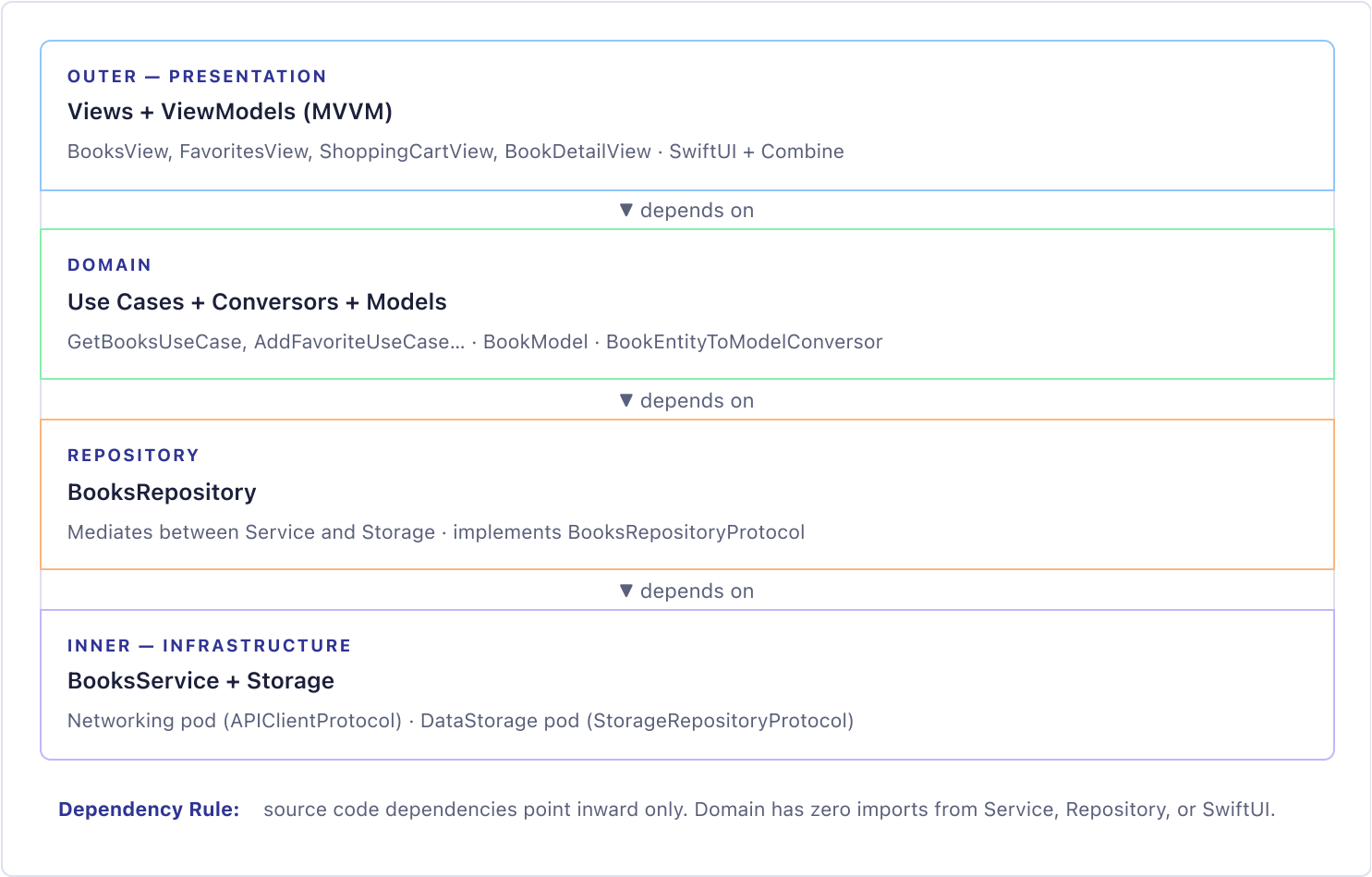
📱 iOS 17+ · SwiftUI · Swift 6 📁 Clean Architecture + MVVM 🟢 52 unit tests · Swift Testing

Overview

BookStore is a SwiftUI iOS application that lists, searches, and manages books from a remote API. Users can mark books as favourites and add them to a shopping cart. All data is persisted locally via SwiftData.

The architecture follows **Clean Architecture** principles — code is divided into concentric layers where inner layers know nothing about outer layers — combined with **MVVM** in the presentation layer to keep views declarative and testable.

Layer Diagram



Layer Breakdown

`BooksService` is the only class that touches the network. It implements `BooksServiceProtocol` — a domain-owned interface that exposes `fetchBooks(page:search:)` returning a plain `BooksModel`. The protocol hides all infrastructure types (`BooksEndpoint`, `APIClientProtocol`) from the layers above.

```
protocol BooksServiceProtocol {
    func fetchBooks(page: Int?, search: String?) async throws -> BooksModel
}
```

REPOSITORY LAYER

`BooksRepository` is the single boundary between Domain and Infrastructure. It owns both the network dependency (`BooksServiceProtocol`) and the persistence dependency (`StorageRepositoryProtocol`), and performs all entity↔model conversions internally via the two Conversor enums. Callers above it speak exclusively in `BookModel` — they never see `BookEntity`.

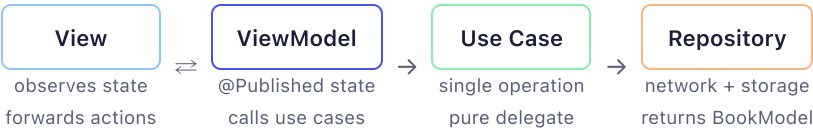
DOMAIN — USE CASES

Each operation in the app maps to one use case class. All nine use cases depend on `BooksRepositoryProtocol`, not on the concrete class. Their execute methods are one-liners that delegate to the repository — they express *what* the app does, not *how*.

Use Case	Signature
GetBooksUseCase	execute(page:search:) async throws -> BooksModel
GetFavoriteBooksUseCase	execute() throws -> [BookModel]
GetCartBooksUseCase	execute() throws -> [BookModel]
AddFavoriteUseCase	execute(model:) throws
DeleteFavoriteUseCase	execute(model:) throws
AddCartUseCase	execute(model:) throws
DeleteCartUseCase	execute(model:) throws
ValidateFavoriteUseCase	execute(model:) throws -> Bool
ValidateCartUseCase	execute(model:) throws -> Bool

PRESENTATION — MVVM

Each screen owns a ViewModel that conforms to an `ObservableObject` protocol (`BooksViewModelable`, etc.). Views observe `@Published` state and forward user actions to the ViewModel — they contain no business logic.



Each screen is assembled by a Factory (`BooksFactory`, etc.) that wires the concrete dependencies together.

Factories are the only place that touches singletons.

Key Design Decisions

1 Protocol-first dependency inversion

Every cross-layer dependency is expressed as a protocol owned by the *inner* layer. This makes every layer independently testable with mock implementations and prevents accidental coupling to concrete types.

2 Repository as the single conversion boundary

Entity↔Model conversion lives exclusively in `BooksRepository`, using `BookEntityToModelConversor` and `BookModelToEntityConversor`. Domain use cases and ViewModels never see `BookEntity`.

3 Intentional singletons with setup injection

`BooksService.shared` and `BooksRepository.shared` are singletons to avoid multiple storage contexts across the app. Dependencies are injected via `setup(base:)` and `setup(schema:service:)` at launch, keeping the class testable (via `internal init`) while keeping the runtime footprint minimal.

4 State enum over booleans

ViewModels expose state as a typed enum (`BooksState`, `FavoritesState`, etc.) with cases `.idle`, `.loading`, `.success([BookModel])`, `.error(String)`. Views pattern-match on this — impossible states (loading + data at the same time) cannot be represented.

5 XCFramework-based dependencies via CocoaPods

Networking and DataStorage are distributed as pre-compiled XCFrameworks through a private CocoaPods spec repository. This keeps the app's source tree clean and allows the pods to evolve independently.

6 Shared SwiftUI components

`FavoritesView` and `ShoppingCartView` share `BookListView`, `BookListRowView`, `BookCoverView`, `BookInfoView`, and `TotalFooterView`. `BookListView` accepts a generic `Footer: View` parameter so each screen can inject its own footer without duplicating list logic.

Testing Strategy

The project uses **Swift Testing** exclusively (no XCTest). All 52 tests are organised by architectural layer and tagged for Xcode's test filter panel. Every layer is tested with mock implementations of the protocols it depends on — no network calls, no real disk storage.

`.SERVICE`

3

`BooksServiceTests`

`.REPOSITORY`

9

`BooksRepositoryTests`

`.USECASE`

9

`UseCaseTests`

`.CONVERSION`

12

`ConversionsTests`

`.VIEWMODEL`

Async ViewModel pattern: `BooksViewModel.fetch()` spawns an internal `Task{}`. Tests annotate the suite with `@MainActor` and call `await Task.yield()` after each `fetch()` to let the internal task complete before asserting state.

Dependency Injection Summary

Every object in the app receives its dependencies through its initializer — there is no global service locator. The chain flows like this at launch:

```
AppConfigurator.configure()  
├─ BooksService.shared.setup(base: url)  
└─ BooksRepository.shared.setup(schema:, service: BooksService.shared)  
  
BooksFactory.create()  
├─ GetBooksUseCase(repository: BooksRepository.shared)  
├─ GetCartBooksUseCase(repository: BooksRepository.shared)  
└─ BooksViewModel(getBooksUsecase:, getCartBooksUsecase:)
```

In tests, concrete classes are replaced by mock implementations (e.g. `MockBooksRepository`) that conform to the same protocols, injected directly via the same initializers.

